

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

**BLM2058 VERİTABANI YÖNETİMİ
DÖNEM PROJESİ RAPORU**

**KYK YURT YÖNETİM SİSTEMİ
İLİŞKİSEL VERİTABANI VE MİMARİSİ**

Hazırlayan
Yusuf Arslan
Öğrenci Numarası: 24290354
2. Sınıf Öğrencisi

Proje Sorumlusu
Dr. Öğr. Üyesi Merve Özkan

Teslim Tarihi: 24.05.2026

1. GİRİŞ VE PROJE TANIMI

Bu rapor, Ankara Üniversitesi Bilgisayar Mühendisliği Bölümü BLM2058 Veritabanı Yönetimi dersi kapsamında oluşturulan 'KYK Yurt Yönetim Sistemi' projesinin teknik mimarisini, veri tabanı mantıksal yapısını, iş kurallarını ve arka plandaki veri süreçlerini kapsamlı bir şekilde sunan dokümandır.

Sistem içerisinde; yeni öğrenci başvurularının alınması, bu başvuruların idari onay süzgecinden geçirilmesi, onaylanan öğrencilerin yurdun durumuna göre uygun odalara yerleştirilmesi, öğrencilerin dönemlik izin haklarının takip edilmesi ve aylık yurt ödemelerinin denetlenmesi gibi kritik süreçler modüler bir mimaride kurgulanmıştır. Geliştirilen sistemi standart veri tabanı panellerinden ayıran temel yönü, sisteme entegre edilen asenkron işlemlerdir. Arka planda multithreading yapısı ile yürütülen bu işlemler, harici bir kullanıcı komutuna gerek kalmaksızın sisteme yapay bir takvim kazandırmakta ve gerçek dünya senaryolarına uygun olarak sürekli veri akışı üretmektedir. Rapor kapsamında, kod karmaşasına yer verilmeksizin tüm sistem soyutlanmış iş mantığı ve mimari katmanlar üzerinden açıklanmıştır.

2. TEKNOLOJİ KATMANLARI VE ÇALIŞMA ORTAMI GEREKSİNİMLERİ

Sistem, sunucu, istemci ve veri depolama katmanlarının birbirinden bağımsız olarak çalışıp birbiriyle asenkron haberleştiği modüler bir yapı üzerine inşa edilmiştir. Veritabanı katmanında ilişkisel veritabanı yönetim sistemi (RDBMS) standartlarına uygun olarak MySQL tercih edilmiş; sunucu tarafında Python ve Flask yapısı kullanılmış; kullanıcı paneli tarafında ise tarayıcı üzerinde doğrudan çalışan HTML5, CSS3 ve Vanilla JavaScript teknolojilerinden yararlanılmıştır.

2.1. Sistem Çalışma Ortamı Özellikleri

Geliştirilen bu proje, XAMPP vb. yerel sunucu ortamlarında test edilerek kararlı çalışması sağlanmıştır. Sunucu katmanında kullanılan Flask framework yapısı ile cihazın yerel MySQL veritabanına sorunsuz bir şekilde bağlantı sağlayabilmesi ve veritabanından dönen verileri anlık olarak dinleyebilmesi için bir veritabanı ara katmanına ihtiyaç duyulmaktadır. Bu bağlantı, Python ile MySQL arasında veri köprüsü kuran özel bir sürücü protokolü XAMPP üzerinden sağlanmıştır.

Bu mimari bağımlılıklardan ötürü, proje dosyaları içerisinde yer alan python, html, css, js ve sql kaynak kodları yerel bir sunucu ve veritabanı katmanı motoru bulunmaksızın doğrudan dosyaya tıklanarak ya da yalıtılmış bir editörde çalışamayacaktır. Kodların çalışabilmesi ve sistemin işlevlerini yerine getirebilmesi için projenin paylaşıldığı GitHub deposundaki yapılandırılmış SQL veritabanı şema dosyasının yerel cihazdaki MySQL motoruna başarıyla aktarılması, sunucu katmanındaki bağlantı parametrelerinin yerel cihaz ortamına göre yapılandırılması ve gerekli Python kütüphane bağımlılıklarının sağlanması gerekmektedir. Bu uygun çalışma ortamı sağlanmadığı sürece katmanlar arası bağlantı kurulamaz ve sistem çalışır hale gelemez.

3. VERİ GEREKSİNİMLERİ VE İŞ KURALLARI

Yurt yönetim süreçlerinin gerçeğe uygun şekilde yürütülmesi, hatalı işlemlerin önüne geçilmesi ve veritabanında tutarsız durumların oluşmasını engellemek amacıyla sistem düzeyinde yapılandırılan temel kurallar şu şekildedir:

- **Odaların Kontenjan Durumu:** Yurttaki odaların blok adı ve oda numarası bazında takibi yapılmaktadır. Her odanın sabit bir yatak kapasitesi bulunur. Odada barınan güncel öğrenci sayısı, odanın maksimum kapasite sınırına ulaştığı anda, bu oda veritabanı filtre sorguları tarafından otomatik olarak 'Dolu' şeklinde işaretlenir. Bu aşamadan sonra yeni öğrencilerin yerleştirilmesi veya mevcut öğrencilerin oda değişikliği talepleri esnasında bu oda bir seçenek olarak arayüzde görüntülenmez.
- **Başvuru ve Kayıt Süreci:** Sisteme iletilen her yeni yurt başvurusu ilk aşamada bekleme durumunda sisteme kayıt edilir. Yurt yöneticisinin bu başvuruyu onaylayarak kalıcı öğrenci kaydına dönüştürebilmesi için veritabanında en az bir adet kapasitesi müsait boş odanın bulunması zorunludur. Yurtta boş yatak kalmaması durumunda, onaylama işlemi iş mantığı katmanı tarafından otomatik olarak engellenir.
- **Referans Bütünlüğü Koruması:** Yurttan kendi isteğiyle ayrılan, mezun olan ya da idari kuralları ihlal ettiği için ilişkisi kesilen öğrencilerin veritabanı kayıtları fiziksel olarak silinmez. Geçmişe dönük idari işlemlerin ve yurt ödemelerinin kayıtlarının referans bütünlüğünü korumak adına, öğrencinin aktiflik durumu pasif konumuna getirilerek kaydedilir. Bu değişiklik işlemiyle birlikte öğrencinin odayla ilişkisi kesilir, odanın doluluk sayısı azaltılır ve öğrencinin henüz onaylanmamış bekleyen tüm izin talepleri iptal edilir.
- **İzin Kullanım Limitleri:** Öğrencilerin yurt mevzuatı gereğince izin hakkı 45 gün olarak belirlenmiştir. Öğrenci arayüzü veya simülasyon üzerinden yeni bir izin talebi girildiğinde, başlangıç ve bitiş tarihleri arasındaki gün farkı hesaplanır. Eğer talep edilen gün süresi öğrencinin mevcut kalan izin hakkından fazla ise işlem sistem tarafından reddedilir. İzin onaylandığında ise ilgili süre kalan haktan otomatik olarak düşülür.
- **Dönemsel Ödeme Kontrolü:** Öğrenciler yurtta kaldıkları her dönem (ay) için belirlenen sabit yurt aidatını ödemekle yükümlüdür. Arka planda işleyen sanal takvime göre her ayın 25. günü geldiğinde sistem o aya ait ödeme kaydı bulunmayan aktif öğrencileri tespit eder ve temsili olarak bildirim gönderir. Sanal zaman akışında yeni aya geçildiğinde, geçmiş aya ait aidat ödemesini gerçekleştirmemiş olan öğrencilerin yurtla ilişkisi sistem tarafından otomatik olarak kesilir ve odalarındaki yerler boşaltılarak yeni başvurulara açılır.

4. VERİTABANI MANTIKSAL TASARIMI VE VERİ SÖZLÜĞÜ (DATA DICTIONARY)

Yurt yönetim sisteminin ilişkisel veri modeli, mantıksal bütünlüğü ve modülerliği sağlamak adına birbirine foreign keys referanslarıyla bağlı 5 ana tablo üzerinde tasarlanmıştır. Veri sözlüğünü oluşturan tablolardaki alan adları, veri tipleri, kısıtlamalar ve açıklamalar, hücre bazında hem yatay hem de dikey olarak tam ortalanmış bir düzen ile aşağıda sunulmuştur:

4.1. Rooms (Odalar) Tablosu

Alan Adı	Veri Tipi	Kısıtlamalar	Açıklama
id	INT	PK, Auto Increment	Odanın benzersiz sistem ID'si
block	VARCHAR(10)	NOT NULL	Odanın yer aldığı bina bloğu
roomno	VARCHAR(10)	NOT NULL	Odanın kapı numarası
capacity	INT	NOT NULL	Maksimum yatak kapasitesi
occupancy	INT	DEFAULT 0	Odadaki mevcut öğrenci sayısı

4.2. Students (Öğrenciler) Tablosu

Alan Adı	Veri Tipi	Kısıtlamalar	Açıklama
tcno	VARCHAR(11)	PK	11 haneli T.C. Kimlik Numarası
name	VARCHAR(50)	NOT NULL	Öğrencinin adı
surname	VARCHAR(50)	NOT NULL	Öğrencinin soyadı
birthdate	DATE	NULL	Sistemce üretilen doğum tarihi
roomid	INT	FK (rooms.id)	Öğrencinin yerleştirildiği oda ID'si
allowance	INT	DEFAULT 30	Kalan izin hakkı gün sayısı
is_active	TINYINT(1)	DEFAULT 1	Yurt kayıt durumu (1: Aktif, 0: Pasif)

4.3. Applications (Başvurular) Tablosu

Alan Adı	Veri Tipi	Kısıtlamalar	Açıklama
tcno	VARCHAR(11)	PK	Başvuran kişinin T.C. kimlik numarası
name	VARCHAR(50)	NOT NULL	Başvuranın adı
surname	VARCHAR(50)	NOT NULL	Başvuranın soyadı
appdate	DATETIME	DEFAULT CURRENT_TIMESTAMP()	Başvurunun sisteme geliş zamanı
status	VARCHAR(20)	DEFAULT 'Pending'	Başvuru onay durumu

4.4. Leaves (İzin Talepleri) Tablosu

Alan Adı	Veri Tipi	Kısıtlamalar	Açıklama
id	INT	PK, Auto Increment	İzin talebi benzersiz kayıt ID'si
tcno	VARCHAR(11)	FK (students.tcno)	Talebi oluşturan öğrencinin TC'si
startdate	DATE	NOT NULL	İzin başlangıç tarihi
enddate	DATE	NOT NULL	İzin bitiş tarihi
status	VARCHAR(20)	DEFAULT 'Pending'	İznin idari onay durumu

4.5. Payments (Ödemeler) Tablosu

Alan Adı	Veri Tipi	Kısıtlamalar	Açıklama
id	INT	PK, Auto Increment	Ödeme işleminin benzersiz makbuz ID'si
tcno	VARCHAR(11)	FK (students.tcno)	Ödemeyi yapan öğrencinin TC'si
period	VARCHAR(7)	NOT NULL	Ödemenin ait olduğu dönem (YYYY-MM)
paydate	DATE	NULL	Ödemenin yapıldığı net tarih
fee	INT	NOT NULL	Tahsil edilen aidat tutarı

5. ASENKRON SİMÜLASYON MOTORU VE ZAMAN AKIŞI MANTIĞI

Sistemin veri tabanı üzerindeki dinamik süreçlerini, yoğun işlem yüklerini ve zaman ayarlı iş kurallarını test edebilmek amacıyla arka planda ana sunucu sürecinden bağımsız çalışan bir simülasyon motoru yapılandırılmıştır. Python'un yerleşik multithreading yetenekleri kullanılarak tasarlanan bu motor, sunucu aktif olduğu andan itibaren bağımsız thread'ler halinde döngüsel işlemler yürütür. Bu sayede sistem, durağan bir yönetim panelinden ziyade kendi takvim döngüsü olan canlı bir sisteme dönüşmektedir. Simülasyon motorunun alt bileşenleri ve mantıksal işleyişi şöyledir:

- **Sanal Zaman Akışı:** Gerçek zaman diliminde geçen her 30 saniyelik süre, sistem içerisinde 1 sanal güne eşitlenmiştir. Sistem genelinde kullanılan “global date” nesnesi bu doğrultuda her 30 saniyede bir gün ileriye taşınır. Zamanın bu şekilde hızlandırılması, normal şartlarda takibi aylar süren süreçlerin (aylık ödemeler, son ödeme günleri, yurt izin talepleri ve kullanımları) çok kısa süreler içerisinde simüle edilmesini pratik hale getirmektedir.

- **Otomatik Başvuru Üretici:** Belirlenen periyotlarla (25 saniye) sistemde kayıtlı olan isim ve soyisim havuzlarından rastgele seçimler gerçekleştirir. Algoritmik olarak kontrol kurallarına uygun geçerli rastgele T.C. Kimlik Numaraları türeterek veritabanındaki başvurular tablosuna başvuru onay durumu “Bekleme” durumunda olan yeni kayıtlar ekler. Bu sayede sürekli onay bekleyen öğrencilerin bulunduğu dinamik bir başvuru durumu oluşturulur.

- **Otomatik İzin ve Ödeme Modülleri:** Sistemdeki aktif öğrenci sayısını belirli aralıklarla kontrol eder. Öğrenci sayısı ile orantılı olacak şekilde çalışarak rastgele aktif öğrencileri seçer ve bu öğrenciler adına sistemde izin talepleri oluşturur. Benzer şekilde, içinde bulunan aktif dönem için henüz ödeme yapmamış olan öğrencileri bularak, rastgele seçilen bireylerin aylık ödemelerini gerçekleştirmelerini sağlamak amacıyla veritabanında “payments” tablosu içinde mevcut dönem için öğrenci ödeme kaydı oluşturur.

6. RESTful API ENDPOINT TASARIMI

Panel arayüzü ile veritabanı katmanı arasındaki tüm veri alışverişi ve iş mantığı kurallarının tetiklenmesi, Flask sunucusu üzerinde yapılandırılmış olan 15 adet RESTful API endpoint üzerinden gerçekleştirilmektedir. Sistem bütünlüğünü yansıtan bu endpointlerin işlevleri ve veritabanı etkileşimleri aşağıda açıklanmıştır:

1. GET /api/date

Sanal zaman motorunun yürüttüğü güncel takvim verilerini istemciye JSON nesnesi olarak aktarır. Bu endpoint'ten dönen veri; YYYY-MM-DD formatında tam tarihi, YYYY-MM formatında güncel dönemi ve kullanıcı panelinde gösterilecek ay/yıl bilgisini içerir. Panel ekranının sağ üst köşesindeki canlı takvim bu endpoint üzerinden periyodik olarak güncellenir.

2. GET /api/notification

Sanal takvim işleyişine göre, içinde bulunulan ayın 25. gününe gelindiğinde o dönem için ödeme yapmayan öğrencilere temsili bir ödeme hatırlatma bildiriminin gönderilir. Ardından ödeme yapmayan öğrenciler içerisinde rastgele sayıda öğrenci seçimi yapılarak mevcut dönem için ödeme yapmış olarak kayıtları güncellenir.

3. GET /api/applications

Veritabanındaki 'applications' tablosunu tarayarak durumu 'Pending' (Beklemede) statüsünde olan tüm yurt başvurularını listeler. Sorgu sonucunda başvuran kişilerin TC kimlik numaraları, ad-soyad detayları ve başvuru yapılma tarihleri asenkron olarak arayüz panelindeki tabloda gösterilir.

4. POST /api/applications/reject

Yöneticinin panel üzerinden belirli bir yurt başvurusunu reddetmesi durumunda tetiklenen endpoint'tir. İstemciden gelen TC kimlik numarası parametresi veritabanında filtrelenerek ilgili başvurunun durum (status) alanı 'Rejected' konumuna getirilir. Reddedilen başvurular için herhangi bir oda ataması veya öğrenci kaydı oluşturulmaz.

5. POST /api/applications/approve

Bekleyen bir başvuruyu kalıcı öğrenci kaydına dönüştürerek yurda yerleştirilmesini sağlar. İş mantığı sürecinde, veritabanında anlık doluluk oranı kapasitesinden küçük olan odalar (yeni bir öğrenci için müsait boş yer olan odalar) taranır ve aralarından rastgele bir oda seçilir. Oda bulunduğu takdirde başvuru durumu 'Approved' yapılır, öğrenci tablosuna yeni satır eklenir ve ilgili odanın doluluk sayısı veritabanında 1 artırılır.

6. GET /api/students

Sistemde kayıtlı olan tüm aktif ve pasif öğrencilerin listesini getiren endpoint'tir. Veritabanı düzeyinde 'students' tablosu ile 'rooms' tablosu oda ID referansları üzerinden JOIN sorgusu ile birleştirilir. Böylece arayüze öğrencinin TC, isim, kalan izin hakkı ve aktiflik durumunun yanında kaldığı odanın blok ve numara bilgisi tek bir seferde iletilir.

7. GET /api/student_detail/<tc>

Belirli bir öğrenciye ait detaylı profil ve geçmiş bilgilerini sorgulamak için kullanılır. URL içerisinde alınan TC kimlik numarası parametresi ile students ve rooms tabloları filtrelenerek JOIN edilir. Öğrencinin doğum tarihi, kalan izin hakkı, oda bilgisi ve ad-soyad detayları profil kartında sunulmak üzere istemci tarafına gönderilir.

8. POST /api/students/update-room

Öğrencinin kalmakta olduğu idari kararla başka bir odayla değiştirmesini sağlayan endpoint'tir. Gelen istek içinde öğrenci TC'si ve geçiş yapmak istediği yeni oda ID'si yer alır. Süreçte öğrencinin eski odasındaki doluluk sayısı 1 azaltılır, öğrenci kaydındaki oda referansı güncellenir ve yeni odanın doluluk sayısı veritabanında 1 artırılır.

9. POST /api/expel

Öğrencinin yurtla ilişkisini kesen ve odasını boşaltan idari endpoint'tir. Gelen TC numarası ile öğrencinin 'is_active' alanı 0 yapılır. Öğrencinin ayrıldığı odanın doluluk sayısı veritabanında 1 düşürülür. Son olarak, veri tutarlılığını korumak adına, bu öğrencinin veritabanında henüz onay bekleyen tüm izin talepleri otomatik olarak iptal konumuna getirilir.

10. POST /api/activate

Daha önce deaktif edilmiş (yurtla ilişkisi kesilmiş) bir öğrencinin yurt kaydını yeniden aktifleştirir. İş mantığı kuralı gereğince; eğer öğrencinin eski odasında hala boş yatak varsa öğrenci o odaya atanır ve odanın doluluğu artırılır. Eski oda tamamen dolmuş ise sistem veritabanındaki müsait odalardan rastgele birini seçerek öğrenciyi yerleştirir.

11. GET /api/rooms

Yurttaki tüm odaların anlık kapasite ve doluluk durumlarını listeler. "rooms" tablosundaki tüm satırlar çekilerek her oda için oda doluluk durumu kontrol edilir. Elde edilen boş yatak sayıları ve doluluk oranları dinamik olarak hesaplanarak arayüz panelindeki görsel progress bar grafikleri ve istatistik kart tasarımları ile ekranda sunulur.

12. GET /api/leaves

Onay bekleyen tüm öğrenci izin taleplerini listelemek için kullanılır. SQL düzeyinde leaves ve students tabloları birleştirilir. Sorgu esnasında SQL `DATEDIFF` fonksiyonu kullanılarak izin başlangıç ve bitiş tarihleri arasındaki net gün süresi hesaplanmıştır. Öğrencinin kalan izin hakkı bilgisiyle beraber arayüzdeki izin onay tablosuna JSON olarak sunulur.

13. POST /api/leaves/reject

Yöneticinin panel üzerinden bir öğrenci izin talebini reddetmesi durumunda çalışan endpoint'tir. Gönderilen izin ID parametresi ile leaves tablosundaki ilgili kaydın onay durumu (status) alanı 'Rejected' olarak güncellenir. Bu işlemde öğrencinin kalan izin hakkı gün sayısı üzerinde herhangi bir eksiltme yapılmaz.

14. POST /api/leaves/approve

Bir öğrenci izin talebini idari olarak onaylayan endpoint'tir. İzin ID'si üzerinden talebin gün süresi ile öğrencinin güncel kalan izin hakkı değeri karşılaştırılır. Kalan izin hakkı yeterli ise talep durumu 'Approved' yapılır ve öğrencinin students tablosundaki kalan izin hakkı gün sayısı, kullanılan izin süresi kadar azaltılarak veritabanında güncellenir.

15. GET /api/payments

İçinde bulunulan aktif sanal döneme ait tüm öğrencilerin aylık ödeme durumlarını sorgular. students tablosu ile payments tablosu o anki aktif sanal dönem parametresiyle `LEFT JOIN` sorgusu üzerinden birleştirilir. Dönemlik ödeme kaydı bulunanlar 'Ödedi', bulunmayanlar ise 'Ödemedi' şeklinde kayıt tablosunda listelenir.

7. KULLANICI ARAYÜZÜ VE ASENKRON İLETİŞİM PRENSİPLERİ

Yurt yönetim panelinin kullanıcı arayüzü, idari personelin sayfayı sürekli yenileme ihtiyacını ortadan kaldıran dinamik bir yapıda, HTML, CSS3 ve Vanilla JavaScript teknolojileriyle geliştirilmiştir. İstisnasız tüm kullanıcı hareketleri ve veri listelemeleri, sunucu katmanıyla asenkron veri transferi mantığına dayalı olarak yürütülmektedir. Fetch API protokolü kullanılarak sunucu endpoint'lerinden çekilen JSON formatındaki ham veriler, tarayıcı tarafında DOM (Document Object Model) ile anlık olarak HTML tablolara, etiketlere vb. dönüştürülür.

Arka planda çalışan asenkron simülasyon motorunun ürettiği anlık verilerin (yeni başvurular, otomatik ödemeler vb.) ekrana eşzamanlı olarak yansması için arayüz tarafında belirli aralıklarla sorgulama yapılmaktadır. JavaScript içerisinde kullanılan zamanlayıcılar belirli aralıklarla sunucunun ilgili GET endpoint'lerini tetikleyerek verileri günceller. Çekilen veriler CSS3 teknolojileri aracılığıyla sayfalarda çeşitli tasarımlarla sunulmuştur.

8. VERİ BÜTÜNLÜĞÜ VE İŞLEM (TRANSACTION) YÖNETİMİ

Simülasyon motorunun sürekli olarak arka planda veritabanına yazma (INSERT/UPDATE) vb. işlemleri gerçekleştirdiği, aynı anda yurt yöneticisinin de panel üzerinden öğrenci ekleme, oda değiştirme veya izin onaylama gibi operasyonlar yürüttüğü eş zamanlı ortamlarda veri bütünlüğünün korunması sistem kararlılığı açısından kritik bir öneme sahiptir. Sistemde bu süreçlerin birbirini bozmasını engellemek ve veritabanındaki oluşabilecek sorunların önüne geçmek amacıyla veritabanı bağlantıları ve işlem sınırları izole bir yapıda tasarlanmıştır.

Veritabanı üzerinde modifikasyon içeren tüm iş mantığı adımları hata yakalama blokları (`try-except`) içerisinde güvenli SQL Transaction sınırları dahilinde yürütülmüştür. Her bir API isteği ve simülasyon iş parçacığı veritabanında kendine ait bağımsız bir oturum (cursor) açar. Bir işlem setinin yürütülmesi esnasında herhangi bir teknik hata oluşması durumunda, yapılan tüm

kısmi deęişiklikler iptal edilerek veritabanı 'rollback' mekanizmasıyla ilk kararlı durumuna geri döndürölür. Tüm adımlar başarıyla tamamlandığında ise veriler 'commit' edilerek veritabanında kalıcı olarak güncellenir. Bu sayede sistemin veri kararlılığı ve tutarlılığı eksiksiz bir şekilde korunmuş olur.

9. SONUÇ VE PROJE ERİŞİM BİLGİLERİ

Bu proje kapsamında, bir devlet yurdunun idari ve operasyonel ihtiyaçlarını karşılamaya çalışan, tüm iş kuralları ve mantıksal kısıtlamaları veritabanı katmanında yapılandırılmış ilişkisel bir yönetim sistemi tasarlanmış ve başarıyla test edilmiştir. Python Flask backend ve yerel MySQL veritabanı entegrasyonu, multithreading bir simölasyon yapısıyla desteklenerek durağanlıktan kurtarılmış ve yaşayan dinamik bir yapı haline getirilerek çeşitlilik katılmıştır.

Geliştirilen KYK Yurt Yönetim Sistemi projesinin çalışan tüm Python backend kaynak kodları, HTML5, CSS3, JavaScript frontend arayüz dosyaları, veritabanı katmanını çalıştırmak için MySQL Workbench üzerinden dışa aktarılan ilişkisel veritabanı şema dosyası (.sql) ve detaylı çalışma ortamı yönergeleri aşağıda adresi belirtilen GitHub deposu üzerinde public olarak paylaşılmıştır:

[GitHub Proje Deposu Erişim Adresi: <https://github.com/yusufarsln/KYK-Yurt-Yonetimi-Sistem-Simulasyonu.git>]